ToolNode와 호출 추적 (실습)

2-1. ToolNode의 역할
2-2. 병렬 도구 호출 (실습)
2-3. 호출 기록 보기 (실습)

2-1. ToolNode의 역할

> 도구 실행을 맡는 ToolNode가 대신해 주는 일을 확인해봅니다.

LangGraph 그래프에서 도구 실행을 담당하는 노드가 ToolNode입니다. LLM이 "이 도구를 이 인자로 호출하겠다"는 tool_calls를 담은 AIMessage를 내놓으면 ToolNode가 받아서 다음을 자동으로 처리합니다.

1. 도구 이름으로 실제 도구 객체 찾기
2. 인자를 넘겨 실행하기 (여러 건이면 병렬로)
3. 결과를 ToolMessage로 만들어 대화 기록(messages)에 추가하기
4. 실행 중 예외가 나면 handle_tool_errors 설정에 따라 처리하기

3일차에서 이 루프를 손으로 짰던 것을 떠올리면 ToolNode가 대신해 주는 일이 정확히 보입니다. 손으로 짠다면 이런 코드가 필요합니다. 지금은 원리를 눈으로 보는 중입니다.

```
from langchain_core.messages import ToolMessage

def tools_node_manual(state: dict) -> dict:
    """ToolNode가 내부에서 하는 일을 손으로 구현한 것 (학습용)."""
    last = state["messages"][-1]
    if not getattr(last, "tool_calls", None):
        return {}

    new_messages = []
    for call in last.tool_calls:
        # 1. 이름으로 도구 찾기
        tool_fn = next((t for t in tools if t.name == call["name"]), None)
        if not tool_fn:
            new_messages.append(ToolMessage(
                content=f"에러: 도구 '{call['name']}'을 찾을 수 없음",
                tool_call_id=call["id"],
            ))
            continue
        # 2. 실행 + 에러 처리
        try:
            result = tool_fn.invoke(call["args"])
        except Exception as e:
            result = f"에러: {e}"
        # 3. ToolMessage로 변환해 추가
        new_messages.append(ToolMessage(content=str(result), tool_call_id=call
["id"]))
    return {"messages": new_messages}
```

이 전체를 다음 한 줄이 대체합니다.

```
from langgraph.prebuilt import ToolNode
ToolNode(tools, handle_tool_errors=True)
```

handle_tool_errors는 세 가지 모드가 있습니다.

설정값	동작	언제 쓰나
True	예외 내용을 ToolMessage로 돌려주고 그래프 계속 진행	운영 기본값
"커스텀 문자열"	지정한 문자열을 ToolMessage로 반환	복구 지침을 통일하고 싶을 때
False	예외를 그대로 던져 그래프 중단	디버깅할 때

주의할 점이 두 가지 있습니다. 먼저 ToolMessage에는 반드시 tool_call_id가 들어가야 합니다. 어느 호출 요청에 대한 응답인지 짝을 맞추는 값입니다. 빠지거나 어긋나면 모델 API가 400 에러를 냅니다. 다만 ToolNode를 쓰면 자동으로 맞춰 줍니다.

그리고 handle_tool_errors를 True로 두면 개발 중 코드 버그까지 조용히 메시지로 바뀌어 원인을 놓치기 쉽습니다. 그래서 개발 초기에는 False로 두고 동작이 안정된 뒤 True로 바꾸는 순서를 권합니다.

에러를 메시지로 바꾸는 위치는 두 곳입니다. 1번째 세션 1-3의 도구 내부 try/except가 더 구체적인 안내를 줄 수 있어 우선입니다. ToolNode의 handle_tool_errors는 최후의 안전망입니다.

> 도구 매칭, 실행, ToolMessage 생성, 에러 처리를 ToolNode 한 줄이 자동화한다는 걸 확인했습니다.

2-2. 병렬 도구 호출 (실습)

> 한 턴에 여러 도구를 동시에 호출하는 경우를 ToolNode가 어떻게 처리하는지 확인해봅니다.

최근 모델들은 한 턴에 여러 도구를 동시에 호출할 수 있습니다. "서울 온도와 부산 온도를 알려줘"라고 물으면 모델이 tool_calls에 두 건을 한꺼번에 담아 내놓고 ToolNode는 두 도구를 병렬로 실행해 ToolMessage 두 개를 돌려줍니다. 왕복 횟수가 줄어 응답이 빨라집니다.

따라하기

1. ToolNode, 병렬 호출, 에러 처리를 한 번에 확인하는 그래프를 만듭니다. 아래 코드를 `day4_practice/toolnode_parallel.py` 로 저장합니다.
   2-3에서 이 파일을 임포트해 다시 씁니다. Bedrock은 답변을 블록 리스트로 주기도 해서 텍스트만 뽑는 `get_text` 함수를 하나 두고 씁니다.

```
# toolnode_parallel.py - ToolNode + 병렬 호출 + 에러 처리 확인
from typing import TypedDict, Annotated, Literal
from dotenv import load_dotenv
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, SystemMessage

load_dotenv()


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


@tool
def get_temperature(city: str) -> str:
    """도시의 현재 온도를 조회한다 (교육용 더미 데이터)."""
    data = {"서울": "3도", "부산": "8도", "제주": "11도"}
    if city not in data:
        raise ValueError(f"{city}는 지원하지 않는 도시입니다. 지원: {list(data)}")
    return f"{city}: {data[city]}"


@tool
def divide(a: float, b: float) -> float:
    """두 수를 나눈다."""
    return a / b     # b가 0이면 ZeroDivisionError, ToolNode가 처리


tools = [get_temperature, divide]


class State(TypedDict):
    messages: Annotated[list, add_messages]


llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
```

```
    region_name="us-east-1",
).bind_tools(tools)


def agent(state: State) -> dict:
    msgs = [SystemMessage(content="도구를 사용해 답하세요.")] + state["messages"]
    return {"messages": [llm.invoke(msgs)]}


def should_continue(state: State) -> Literal["tools", "end"]:
    return "tools" if state["messages"][-1].tool_calls else "end"


builder = StateGraph(State)
builder.add_node("agent", agent)
builder.add_node("tools", ToolNode(tools, handle_tool_errors=True))
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", should_continue, {"tools": "tools",
"end": END})
builder.add_edge("tools", "agent")
graph = builder.compile()


if __name__ == "__main__":
    # 1) 병렬 호출: 한 질문에 도구 두 개가 한 턴에 호출됨
    print("=== 병렬 호출 ===")
    result = graph.invoke({"messages": [HumanMessage(content="서울과 부산의 온도
를 동시에 알려주세요.")]})
    for m in result["messages"]:
        if getattr(m, "tool_calls", None):
            for tc in m.tool_calls:
                print(f"  도구 호출 요청: {tc['name']}({tc['args']})")
        elif m.type == "tool":
            print(f"  도구 결과: {get_text(m)}")
    print(f"최종 답변: {get_text(result['messages'][-1])}")

    # 2) 에러 처리: 0으로 나누기 -> ToolMessage로 변환되어 Agent가 복구
    print("\n=== 에러 처리 (handle_tool_errors=True) ===")
    result = graph.invoke({"messages": [HumanMessage(content="10 나누기 0
은?")]})
    print(f"최종 답변: {get_text(result['messages'][-1])}")
```

2. 실행해서 첫 질문에서 도구 호출 요청 두 건이 한 턴에 찍히는지 봅니다. 병렬 호출 턴의 AIMessage 내부는 이런 모양입니다. tool_calls 리스트에 두 건이 함께 들어 있는 것이 핵심입니다.

```
AIMessage(tool_calls=[
    {"name": "get_temperature", "args": {"city": "서울"}, "id": "call_abc"},
    {"name": "get_temperature", "args": {"city": "부산"}, "id": "call_def"},
])
```

막힐 때

- 병렬 호출이 안 나오고 도구를 한 번씩 두 턴에 걸쳐 부릅니다.

  정상입니다. 병렬 여부는 모델의 판단이라 강제할 수 없습니다. 질문을 "두 도시를 동시에"처럼 명확히 하면 병렬로 나올 확률이 높아집니다.

- ThrottlingException이 납니다.

  분당 호출 한도에 걸린 경우입니다. 몇 초 기다렸다 다시 실행합니다.

- 도구 함수 안에서 전역 변수나 파일을 수정한다면 병렬 실행 때 경쟁 상태(두 실행이 같은 값을 동시에 건드려 결과가 꼬이는 문제)가 생길 수 있습니다. 조회 도구는 상관없지만 쓰기 도구는 병렬 실행을 염두에 두고 설계합니다.

확인: 첫 질문에서 도구 호출 요청 두 건이 한 턴에 찍히고 0으로 나누기 질문에서 그래프가 죽지 않고 자연어 답변이 나오면 성공입니다.

> 모델이 한 턴에 여러 도구를 호출하고 ToolNode가 병렬 실행과 에러 변환을 처리한다는 걸 확인했습니다.

2-3. 호출 기록 보기 (실습)

> 어떤 도구가 어떤 인자로 불렸는지 trace를 출력해 실행 과정을 추적해봅니다.

"Agent가 무슨 도구를 어떤 인자로 불렀고 무엇을 받았는가"를 확인하는 작업을 trace(추적)라고 합니다. 오늘 마지막 도전 미션에서 장애 지점을 찾는 수단이므로 여기서 확실히 익혀 둡니다. 방법은 두 가지입니다.

1. **코드로 직접 확인**: 최종 state의 messages 리스트를 순회하면 전체 히스토리가 그대로 남아 있습니다. 별도 도구 없이 즉시 가능하고 로그로 남기기도 좋습니다. 이번 과정의 기본 방법입니다.
2. **LangSmith 사용**: 환경 변수만 설정하면 모든 실행이 웹 대시보드에 기록됩니다. 사내 보안 정책상 못 쓰는 환경이 있으므로 참고로만 소개하겠습니다.

따라하기

1. 2-2의 그래프를 임포트해 trace 출력 함수와 복합 질문을 실행하는 새 파일을 만듭니다. 아래 코드를 `day4_practice/trace.py` 로 저장합니다.

```
# trace.py - messages 리스트를 순회하며 실행 과정을 표 형태로 출력
import json
from langchain_core.messages import HumanMessage
from toolnode_parallel import graph   # 2-2에서 만든 그래프를 그대로 사용


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinstance(block, dict))
    return content


def print_trace(result: dict):
    """messages 리스트를 순회하며 실행 과정을 표 형태로 출력."""
    print(f"{'순서':<4} {'타입':<10} {'내용'}")
    print("-" * 70)
    for i, m in enumerate(result["messages"]):
        if m.type == "human":
            print(f"{i:<4} {'사용자':<10} {get_text(m)}")
        elif m.type == "ai" and getattr(m, "tool_calls", None):
            for tc in m.tool_calls:
                print(f"{i:<4} {'도구요청':<10} {tc['name']} args={json.dumps(tc['args'], ensure_ascii=False)}")
        elif m.type == "tool":
            print(f"{i:<4} {'도구결과':<10} [{m.name}] {get_text(m)[:60]}")
        elif m.type == "ai":
            print(f"{i:<4} {'최종답변':<10} {get_text(m)[:60]}")


if __name__ == "__main__":
    result = graph.invoke({"messages": [HumanMessage(content="서울 온도를 100으로 나눈 값은?")]})
    print_trace(result)
```

2. 실행하면 도구가 이어 불린 순서가 표로 나옵니다.

```
순서  타입         내용
------------------------------------------------------------------------
0    사용자       서울 온도를 100으로 나눈 값은?
1    도구요청     get_temperature args={"city": "서울"}
2    도구결과     [get_temperature] 서울: 3도
3    도구요청     divide args={"a": 3, "b": 100}
4    도구결과     [divide] 0.03
5    최종답변     서울의 온도 3도를 100으로 나누면 0.03입니다.
```

3. (참고) LangSmith를 쓸 수 있는 환경이라면 `.env`에 다음을 추가하는 것만으로 모든 실행이 웹 대시보드에 기록됩니다. 코드 수정은 필요 없습니다.

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_...
LANGSMITH_PROJECT=day4-practice
```

막힐 때

- `from toolnode_parallel import graph`에서 임포트 에러가 납니다.

  두 파일이 같은 폴더(day4_practice)에 있는지 확인합니다.

- 임포트하는 순간 2-2의 테스트까지 같이 실행됩니다.

  2-2 파일의 실행 코드가 `if __name__ == "__main__":` 블록 안에 있는지 확인합니다. 이 블록은 파일을 직접 실행할 때만 돌고 임포트할 때는 돌지 않습니다.

- 중간 과정을 실시간으로 보고 싶습니다.

  `graph.stream(..., stream_mode="updates")`를 쓰면 실행 도중에도 노드별 업데이트를 볼 수 있습니다. 5번째 세션 5-2에서 직접 씁니다.

확인: 첫 도구의 결과(3도)가 두 번째 도구의 인자(a=3)로 이어지는 흐름이 trace 표에서 읽히면 성공입니다.

> messages 순회만으로 도구 호출 순서와 인자를 추적할 수 있다는 걸 확인했습니다.